

category: dotnet-general

tags: [wsl, linux, dotnet-publish, projectreference, absolute-path, relative-path, msbuild, build-deb]

severity: high

created: 2026-07-24

updated: 2026-07-24

project-origin: iPGSv4 (IPGSUseCam)

ProjectReference tuyệt đối `E:\KZTEK\...` chặn build .deb qua WSL — sửa sang tương đối AN TOÀN khi cùng 1 máy dev, đã verify cả 2 OS

Tình huống gặp phải

Chạy `scripts/linux-deb/build-deb.sh` (đã có sẵn, không sinh mới) cho `IPGSUseCam` (Avalonia) — gọi qua `wsl -e bash -lc "... dotnet publish ..."` từ Git Bash trên máy dev Windows, dùng `dotnet` cài sẵn trong WSL Ubuntu (`~/dotnet/dotnet`, không có trong PATH mặc định).

Triệu chứng / Lỗi

```
Skipping project
".../IPGSUseCam/E:/KZTEK/Code_Git/5.BaseUI/KztekComponentAvalonia/.../KztekComponentAvalonia.csproj" because it was not found.
...
error CS0246: The type or namespace name 'KztekComponentAvalonia' could not be found
error CS0246: The type or namespace name 'KzButton' could not be found
error CS0246: The type or namespace name 'KzCheckedListBox' could not be found
```

Lỗi lan ra nhiều project downstream (`IPGS.Control`, `IPGSUseCam`, `ApplicationConfig`, `Kztek.Cameras.Avalonia` ở repo `0.BaseLIB` khác) — dễ nhầm tưởng thiếu package hoặc lỗi XAML, nhưng gốc rễ chỉ ở 1 dòng `ProjectReference`.

Nguyên nhân gốc rễ (Root Cause)

4 file `.csproj` dùng `ProjectReference`
`Include="E:\KZTEK\Code_Git\5.BaseUI\KztekComponentAvalonia\KztekComponentAvalonia\KztekComponentAvalonia.csproj"` (đường dẫn Windows tuyệt đối, cố ý thêm ngày 2026-07-23 để "hợp nhất về kho dùng chung"). `dotnet` chạy trong WSL/Linux không hiểu `E:\...` là ổ đĩa tuyệt đối (Linux không có khái niệm ổ đĩa, backslash không phải separator) → MSBuild nối nhầm thành path con nằm dưới project hiện tại → restore "Skip" project đó trong im lặng → mọi type từ `KztekComponentAvalonia` biến mất ở toàn bộ project tham chiếu.

Giải pháp

Đổi cả 4 `ProjectReference` sang đường dẫn tương đối trở đúng

`Code_Git\5.BaseUI\KztekComponentAvalonia\...`:

```
<!-- IPGS.Control/IPGS.Controls.csproj, IPGSUseCam/IPGSUseCam.csproj,
ApplicationConfig/ApplicationConfig.csproj -->
<ProjectReference
Include="..\..\..\..\..\5.BaseUI\KztekComponentAvalonia\KztekComponentAvalonia\Kz
tekComponentAvalonia.csproj" />

<!-- 0.BaseLIB/Kztek.Camera/.../Kztek.Cameras.Avalonia.csproj (repo khác, độ sâu khác)
-->
<ProjectReference
Include="..\..\..\..\..\5.BaseUI\KztekComponentAvalonia\KztekComponentAvalonia\Kztek
ComponentAvalonia.csproj" />
```

1. Đếm đúng số cấp thư mục từ vị trí `.csproj` tới `Code_Git` bằng cách test thực tế (`cd <thư mục .csproj> && ls "<path tương đối>"` trước khi sửa) — KHÔNG đếm bằng mắt trên string path, rất dễ lệch 1 cấp (đã lệch 1 lần ở `Kztek.Cameras.Avalonia.csproj`: viết `5 ..\` thay vì `6`, gây lỗi `Skipping project ".../1.Window/5.BaseUI/..."`).
2. Build lại qua WSL để xác nhận hết lỗi.
3. **Bắt buộc build lại trên Windows** (`dotnet build <mỗi csproj đã sửa>`) để xác nhận không phá build hiện tại — đây là bước verify quan trọng nhất, xem mục Gotchas.

Áp dụng lại (How to reuse)

- Thấy `Skipping project ".../<tên project hiện tại>/<ô đĩa>:/..."` trong log restore → chắc chắn là `ProjectReference` tuyệt đối kiểu Windows không resolve được trên Linux/WSL, sửa ngay không cần đoán thêm.
- Trước khi sửa path tương đối, luôn `cd` vào thư mục chứa `.csproj` và `ls` thử path tương đối để xác nhận đúng số cấp — không suy luận bằng cách đếm chữ trên màn hình.
- Sau khi sửa, chạy build cả 2 phía (WSL và Windows) trước khi coi là xong — không chỉ tin vào 1 phía.

Chú ý / Cạm bẫy (Gotchas)

- ⚠️ **Đổi lập có chủ đích với lesson wsl-hybrid-build-windows-projectref-msys-pathconv.md**: lesson đó cảnh báo "đừng sửa `ProjectReference` tuyệt đối sang tương đối nếu 2 repo không thực sự nằm cùng cấu trúc tương đối trên MỌI máy dev". Ở case này, việc sửa AN TOÀN vì: (1) chỉ có 1 máy dev duy nhất đang dùng repo `KztekComponentAvalonia` dùng chung tại `Code_Git\5.BaseUI`, (2) cấu trúc thư mục `Code_Git` cố định trên máy đó cho mọi repo con, (3) đã build lại và verify **0 Error** trên cả Windows lẫn WSL sau khi sửa. Nếu môi trường có NHIỀU máy dev với cấu trúc `Code_Git` khác nhau → PHẢI dùng giải pháp build-script (publish trên Windows trước) như lesson kia, KHÔNG sửa path.
- ⚠️ Dễ lệch 1 cấp `..\` khi tính bằng mắt — luôn verify bằng `ls` thực tế trước khi Edit, và build lại ngay sau khi sửa để bắt lỗi "Skipping project" sớm (log restore, không phải log compile).
- ⚠️ Comment cũ trong file giải thích lý do đổi sang absolute path ("hợp nhất về kho dùng chung", 2026-07-23) — nghĩa là có quyết định trước đó chủ đích dùng absolute. Trước khi đổi ngược lại, nên hiểu lý do gốc (thường là để tránh lệch path khi 2 repo không cùng cấu trúc) rồi xác nhận với người quyết định nếu có thể, thay vì âm thầm revert.

Tham chiếu

- Project liên quan: `iPGSv4` — `IPGS.Control/IPGS.Controls.csproj`, `IPGSUseCam/IPGSUseCam.csproj`, `ApplicationConfig/ApplicationConfig.csproj`, `0.BaseLIB/Kztek.Camera/.../Kztek.Cameras.Avalonia.csproj`, `scripts/linux-deb/build-deb.sh`
- Lesson liên quan: `[wsl-hybrid-build-windows-projectref-msys-pathconv.md]`(`wsl-hybrid-build-windows-projectref-msys-pathconv.md`) — cùng chủ đề `ProjectReference` tuyệt đối chặn build WSL, nhưng giải pháp khác (không sửa path) vì bối cảnh nhiều máy dev
- Lesson liên quan: `[wsl-linux-deb-build-global-json-sdk-version-mismatch.md]`(`wsl-linux-deb-build-global-json-sdk-version-mismatch.md`)